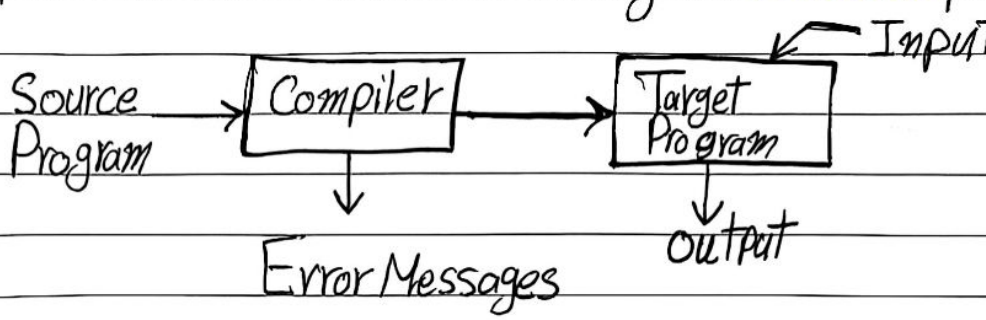
Compiler

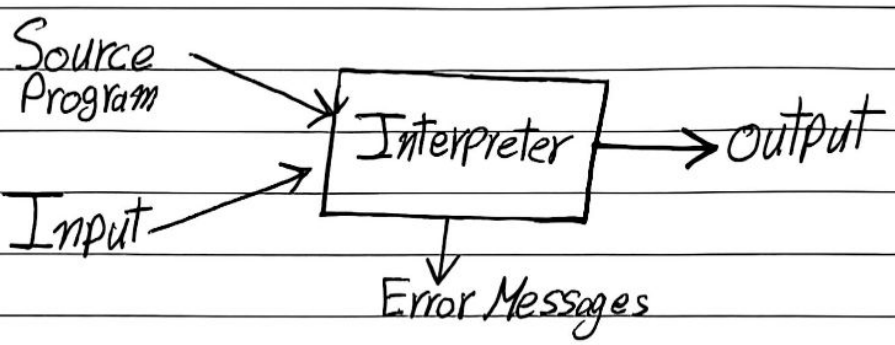
What is a Compiler?

A compiler is a program that can read a program in one language (the Source language) and translate it into an equivalent program in another language (the Target language).
It also reports the errors it detects during the translation process



What is an Interpeter?

It is another kind of language processor.
Instead of translating code into a target program, it directly executes the Source code, processing User input as it runs

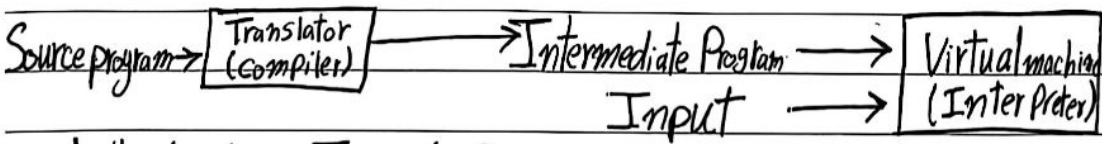


Compare between Compiler & Interpreter

	Compiler	Interpreter
1- Input:	Takes entire program as input	Takes Single instruction as input
2- Speed:	Faster	Slower
3- Intermediate code:	Generated	Not generated
4- Memory:	Requires More memory due to intermediate object code	Requries Less Memory as there is no intermedicite object code
5- Program:	not Compiled every time	Every time higher level program is converted into lower level program
6- Errors:	Displayed after entire Program is checked	Displayed for every instruction interpreted
7- Debugging:	Hard	Easy
8- Examples:	C, C++.	Python, Ruby

What is a Hybrid Compiler?

It combines compilation and interpretation, the compiler produces intermediate code which is then interpreted ~~rather~~ instead of being compiled to machine code, Ex: Java



What does Java do?

It combines both compilation and interpretation (Hybrid Program), by first ~~translating~~ compiling Source code to bytecodes, which are then interpreted by virtual machine(JVM)

What are byte codes? Intermediate representation, not human readable code, not machine code but something in between that a Virtual machine can interpret.

★ السؤال بتيجي على انو احنا نرسم الـ Phases ونحل مسألة عليها بيدعاني

① ★ Phases of compiler translation: أول خطوة: نرسم الرسمة الكبيرة

Character Stream

↓

lexical analyzer

↓ token Stream

Syntax analyzer

↓ syntax tree

Semantic analyzer

↓ syntax tree

intermediate Code generation

↓ intermediate representation

machine independent Code optimizer

↓ intermediate representation

Code generator

↓ target machine code

machine dependent Code optimizer → target machine code

Step ②: translation المرحلة

Ex: Tax = (Sal - deduct) * Percent

A ← نرسم الرسمة الكبيرة

B ← نمشي على كل خطوة في الرسمة

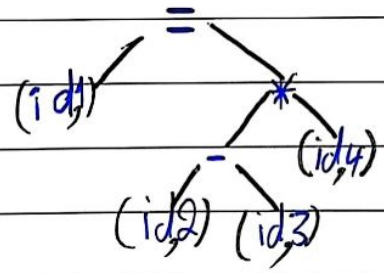
① lexical analyzer

<id,1>, <=>, <(>, <id,2>, <->, <id,3>, <)>, <*>, <id,4>

هنا بنلاقي كل الـ variable ونديلها id

② Syntax analyzer:

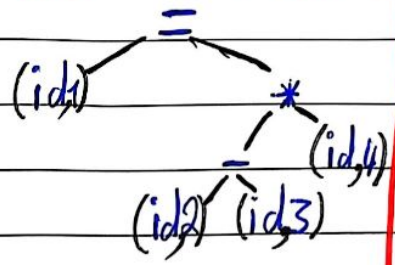
Binary tree بنعمل



③ Semantic analyzer

بنعمل type casting لو مثلا ... ج

ومبيقى عندو Binary tree بردو



④ intermediate code gen.

1- $t_1 = id2 - id3$

2- $t_2 = t_1 * id4$

3- $id1 = t_2$

★ كل عملية تتكتب في سطر

⑤ machine indep. code opt.

لو الـ assign على متغيرين تعملها في سطر واحد

1- $t_1 = id2 - id3$

2- $id1 = t_1 * id4$

★ أو هي تدمج العمليات السابقة

⑥ Code generator

★ بنعمل ASM code ونعرف الـ source والـ dest.

op	source	dest
LDA	id2,	R1
LDA	id3,	R2
SUB	R1,	R2 R3
LDA	id4,	R4
MUL	R4,	R3, R5
STR	id1,	R4

① Lexical Analyzer (Scanner):

It is the first Phase of a Compiler, it reads characters from Source code, groups them into lexemes, and outputs tokens for the parser (syntax analyzer)

Roles & Tasks

1. Main task: Read Characters → Group into lexems → output tokens to Syntax analyzer
2. Strip: Removes whitespaces (Spaces, tabs, newlines) and comments
3. Track lines: Count newlines to report error location
4. Symboltable: Insert identifiers into the table; Sometimes read from it to determine token type

Types of Process:

1. Scanning : Removes comments and compacts whitespace (no tokens yet)
2. Tokenization: Produces the actual sequence of tokens

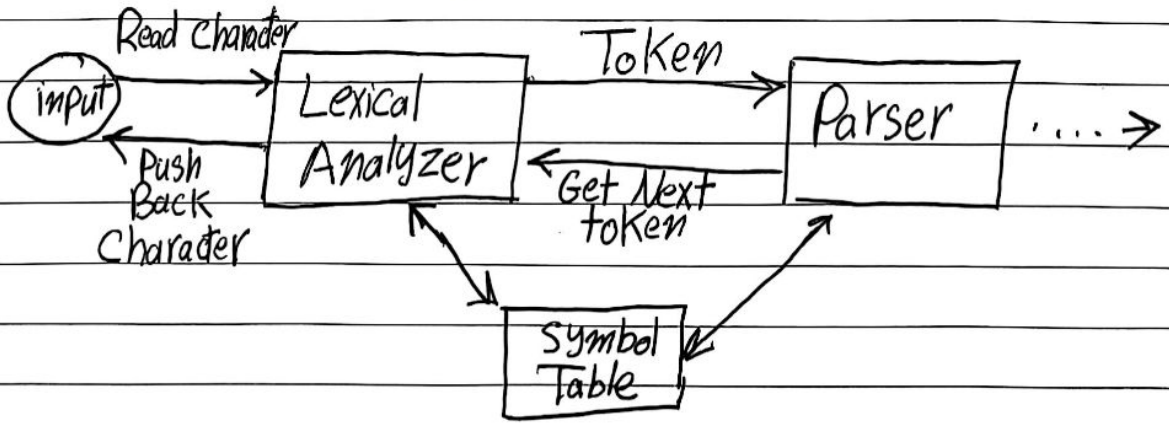
lexeme: The actual character sequence in the Source code that matches a pattern

Pattern: A rule describing what lexemes look like for a token (eg "id = letter followed by letters/digits").

Token: A pair: (token name, attribut value), it is the output of lexer, the attribute is optional, Keywords and operations don't need one, but id and literal do.

Example:

Sample Lexems	Pattern	Token
if	Characters i, f	if
Pi, Score, D2	Letter followed by letters or digits	id



② Syntax Analyzer:

It is the second phase of a compiler, it tokens from the lexer and builds a tree like-intermediat representation (Parse tree) that shows the grammatical structure of the code.

③ Semantic analyzer:

It is the third phase of a compiler, it uses the syntax tree for logical correctness (type matching, declarations, etc.) using the symbol table for Semantic consistency with the language definition.

Formal Language Theory:

Alphabet: A finite set of symbols (e.g., $\{a, b, c\}$ or $\{0, 1\}$)

String (Word, Sentence): A finite Sequence of symbols from an alphabet. Length $|S|$ = number of symbols

Empty String (ϵ or λ): The String with zero Symbols. Length = 0.

Language: A set of Strings over an alphabet

Empty Language (ϕ): A language with NO strings at all Not even the empty string $\phi = \{\}$

Language $\{\lambda\}$: A language containing only one String: the empty String $\{\lambda\} = \{""\}$
Note: $\phi \neq \{\lambda\}$.

Parts of Strings (Example: Banana)

Term	Definition	Example
Prefix	String obtained by Removing Symbols from the END.	λ, anana, na
Suffix	String obtained by removing symbols from the begging.	λ, B, Ba
Substring	String obtained by removing from both ends.	λ, ana
Subsequence	String obtained by removing any position (can Skip).	Baan, bn
Proper	Excludes the λ and the full String itself.	Proper prefix of Banana = B, Ba, Ban, Bana, Banan, ~~Banana~~

أي نافي الـProper مش بغرض وجود λ بعد أو قبل الـ String الأصلي

String operations:

Concatenation(xy): Glues strings together

Exponenation(s^n): Repeat string n times

Regular Expression:

A language is regular if it can be built using only: Union, Concatenation, and Kleene Star.

Context Free Grammar:

A set of recursive rewriting rules used to specify the syntax of a language.
It describes the hierarchial structure of most programming languages.
It has 4 components: $G = (N, T, S, P)$ where...
N: Set of Non-terminals (variables)
T: Set of terminals
S: Start Symbol
P: Set of Production rules
and Regex $\subset$ CFG which means any valid regex is CFG but not the other way.

Transition diagram:

A directed graph representing a finite automation (DFA/NFA)

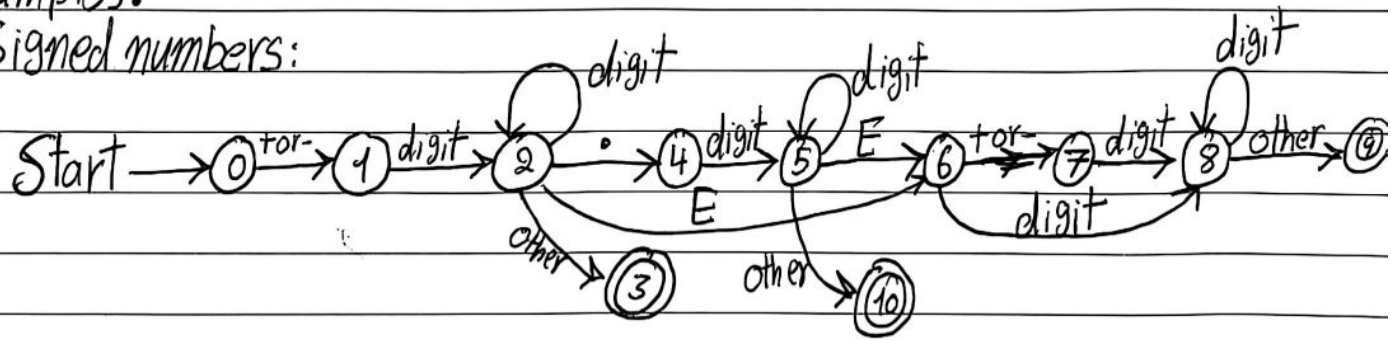
It consists of:

1. State: Circle (node)
2. Transition: Arrow from State to State, labeled with input symbol
3. Start State: Arrow pointing to it with no source
4. Accept/Final State: Double circle



Examples:

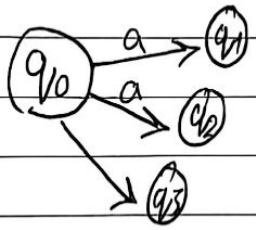
Signed numbers:



Finite Automata

NFA: (Nondeterministic Finite Automata)

Can have multiple transitions on the same input symbol, leading to different states and allows ε-transitions (moves without reading [consuming] input).



DFA: (Deterministic Finite Automata)

Exactly one transition per input symbol for each state. No ε-transitions



NFA to DFA algorithm:

```
S = S0 ;                  //Start at initial state
C = nextchar( );          // read first input character

while (C !=eof) {         //while there is still input
    S= move(S, C);        // Follow transition from state S on input C
    C = nextChar( );      // Read next character
}

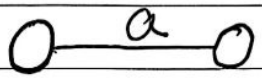
if (S is in F)            //if Final state is an accepting State
    return "yes";         //String accepted
else
    return "no";          //String rejected
```

Thompson

Compiler week 5

Drawing rules:

① a



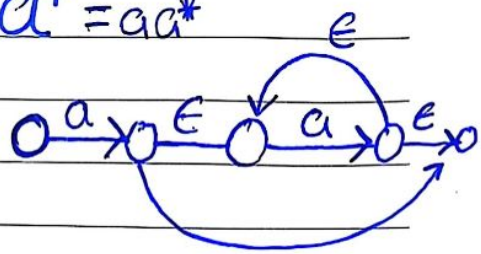
② ab



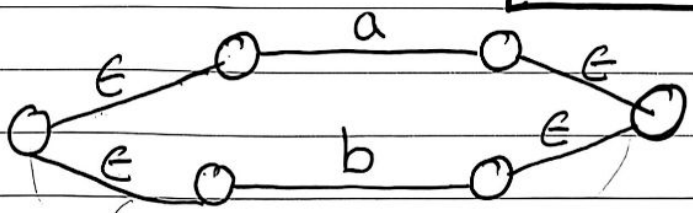
⑥ ★ Trick

علاكتفل a واحدة ...

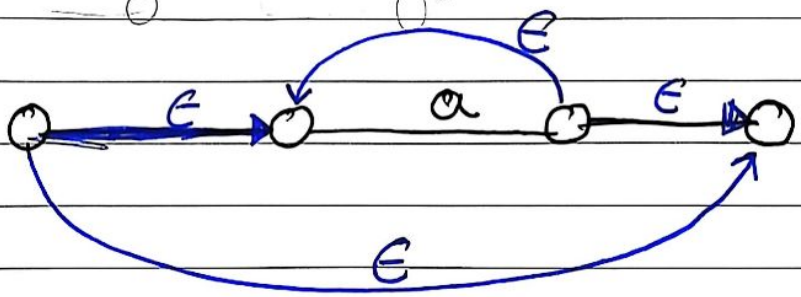
$a^+ = aa^*$



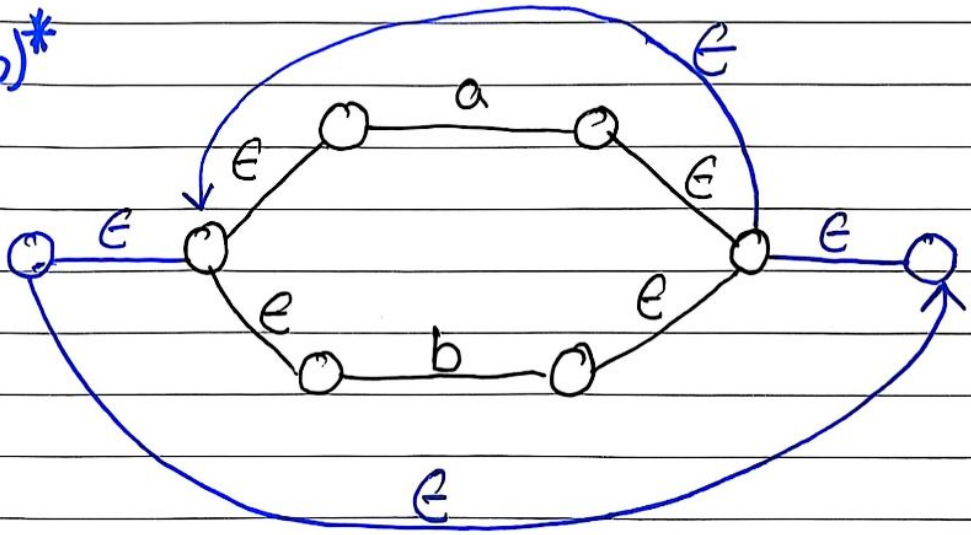
③ a|b



④ a^*



⑤ $(a|b)^*$

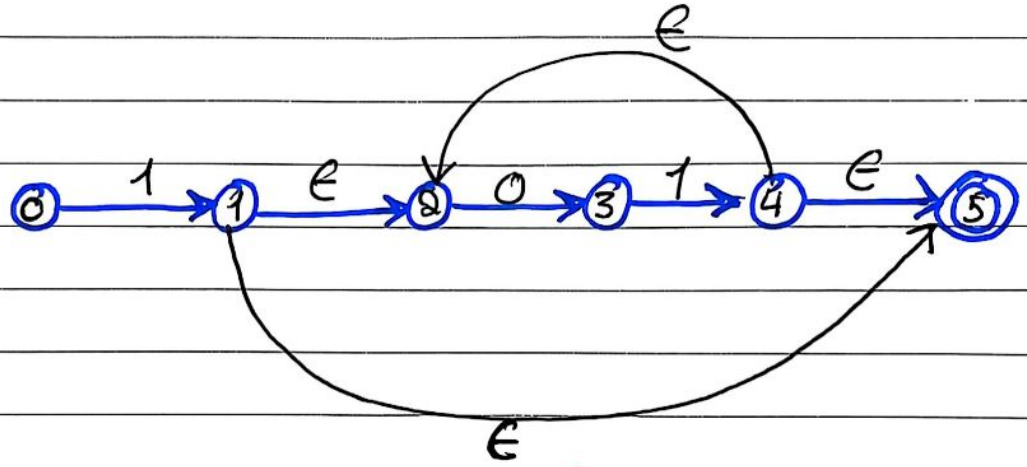


NFA → DFA

ϵ-closure
⇓
Transition table
⇓
Drawing
⇓
DFA minimization / optimization

☆عند كل State بمشي ورا الϵ في الطريق بناعها...

① ϵ-closure



ϵ-closure: ب ϵ-closure أول شي هنسي → State A

ϵ-closure(0) = {0}
ϵ-closure(1) = {1, 2, 5}
ϵ-closure(2) = {2}
ϵ-closure(3) = {3}
ϵ-closure(4) = {2, 4, 5}
ϵ-closure(5) = {5}

★ كل الـ States اللي طالع فيها أخر State هاخدها accepted States

② Transition table:

Ⓐ جهز الجدول

Ⓑ ★ move(A, x / y / z ...)

Ⓐ → جهز الجدول

Ⓑ ★ move

كل State جديدة من الـ ϵ-closure بديلها حرف جديد.

① Start with Ⓐ

★ move(A, 0) = ϕ

★ move(A, 1) = {1} ← هاخد ϵ-closure ليها

ϵ-closure(1) = {1, 2, 5} ⇒ Ⓑ

Transition table

State	0	1
A {0}	ϕ	B
B {1, 2, 5}	C	ϕ
C {3}	ϕ	D
D {2, 4, 5}	C	ϕ

② Ⓑ

★ move(B, 0) = {3}

⇒ ϵ-closure(3) = 3 ⇒ Ⓒ

★ move(B, 1) = ϕ

③ Ⓒ

★ move(C, 0) = ϕ

★ move(C, 1) = {4}

∴ ϵ-closure(4) = {2, 4, 5} ⇒ Ⓓ

Continue

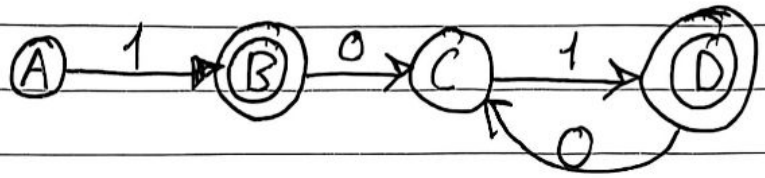
④ Ⓓ

★ move(D, 0) = {3} = C

★ زي ما أنا عارف ان ϵ-closure بتاع 3 = C

★ move(D, 1) = ϕ

③ DFA Drawing



④ DFA optimization (على مسؤليتي تانية في الطاقة)

1. acceptance States & non-acceptance states
2. Transition table of the Drawing
3. The Splitting table → ☆الاسم على الـ non-accepting
   عن طريق بشوف اي حاجة في الـ States باختي
4. The new transition table (minimized)
5. The new optimized drawing

① acceptace = {E}
Non-acceptance = {A, B, C, D}

② Transition Table:

State	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

③ The splitting Table

	Split on a	Split on b
{A, B, C, D} {E}	{A, B, C, D} {E}	{A, B, C} {D}, {E}
{A, B, C} {D} {E}	No split	{A, C} {B} {D} {E}
{A, C} {B} {D} {E}	No split	No Split

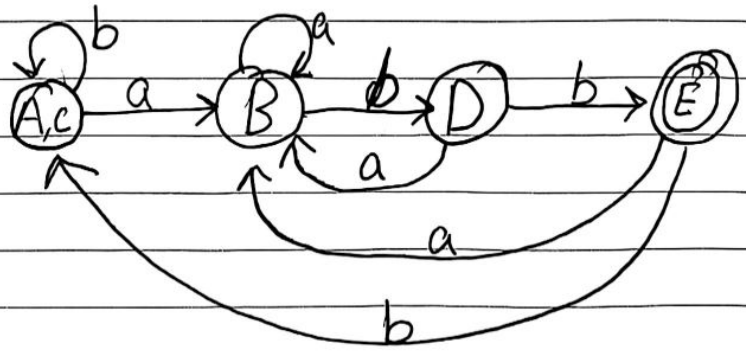
☆ Trick: احنا مش بنقسم بس لو في Set مختلفة يعني ممكن نلاقي Split على Set تانية مع بعضها فـ.. شرح بعدين بقى المهم تفضل فضلك

④ Optimized Transition Table

خلاص بقى الـ A, C بعوا نفس الحاجة

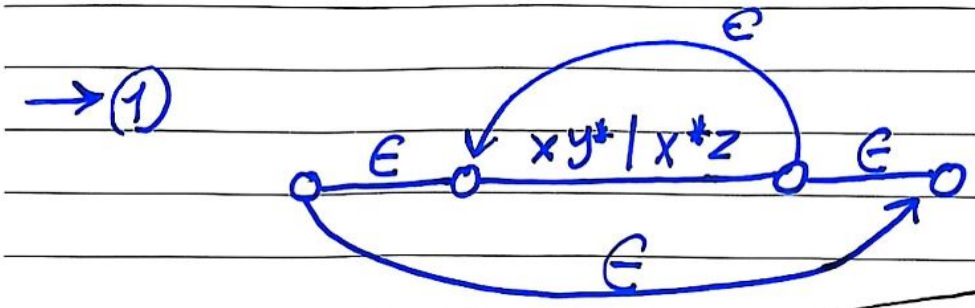
	a	b
A, C	B	A, C
B	B	D
D	B	E
E	B	A, C

⑤ Draw:

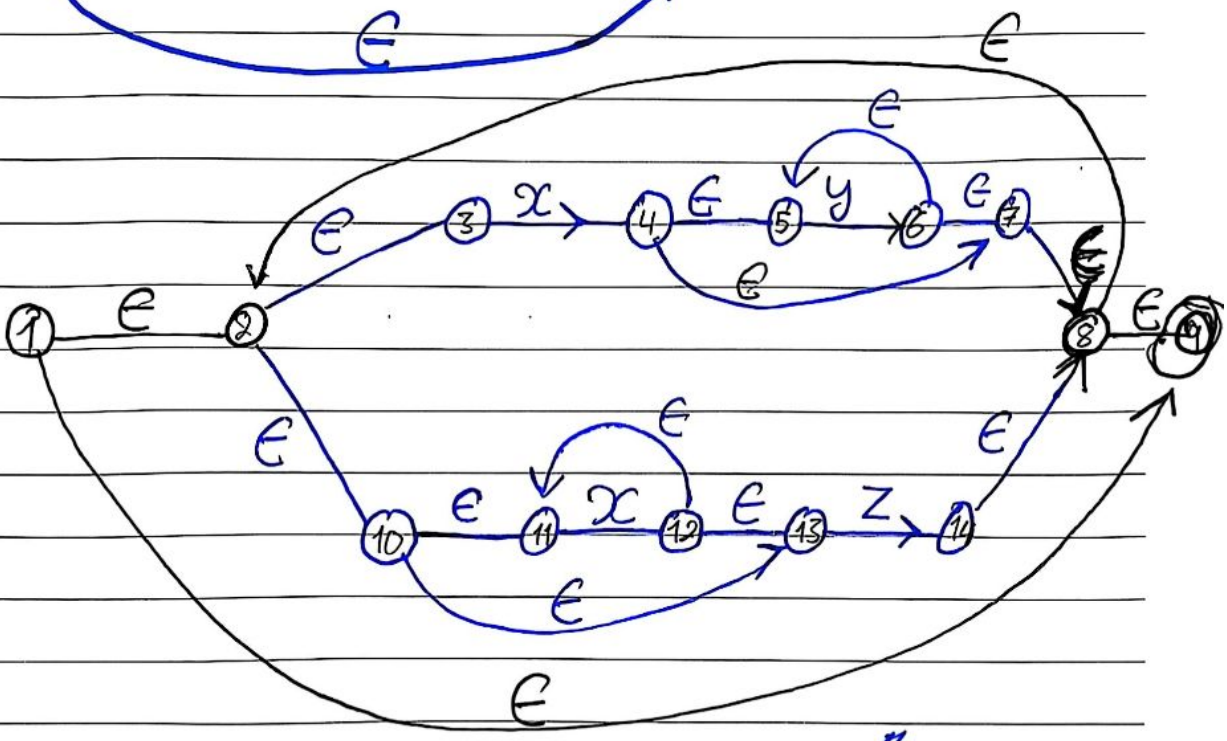


★ Convert to NFA:

$$(xy^* \mid x^*z)^*$$



(2)



≠

Convert to DFA:

ϵ-closure:

1 = {1, 2, 3, 10, 11, 13, 9} ← State A

mov(A, x) = {12, 4}

ϵ-closure = {12, 11, 13, 4, 5, 7, 8, 2, 3, 10, 9} ← State B

mov(A, y) = ∅

mov(A, z) = {14}

ϵ-closure = {14, 8, 9, 2, 3, 10, 13, 11} ← State C

mov(B, x) = {12, 4} ← A

mov(B, y) = {6}

ϵ-closure = {6, 5, 7, 8, 9, 2, 3, 10, 11, 13} ← State D

mov(B, z) = {14} ← C

mov(C, x) = {12, 4} ← A

mov(C, y) = ∅

mov(C, z) = {14} ← C

mov(D, x) = {12, 4} ← A

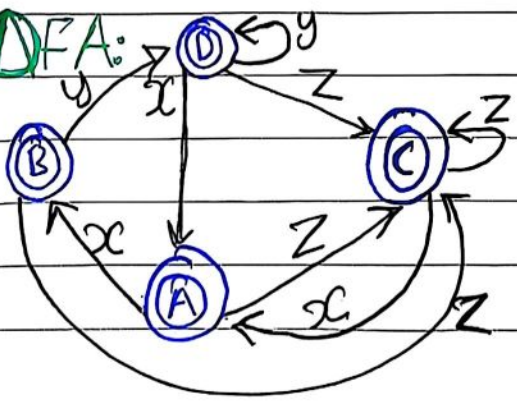
mov(D, y) = {6} ← D

mov(D, z) = {14} ← C

Transition Table:

State	X	Y	Z
A{1, 2, 3, 10, 11, 13, 9}	B	ϕ	C
B{2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13}	A	D	C
D{2, 3, 5, 6, 7, 8, 9, 10, 11, 13}	A	D	C
C{2, 3, 8, 9, 10, 11, 13, 14}	A	ϕ	C

DFA:



DFA optimization:

① acceptance: {A, B, C, D}

② Transition Table

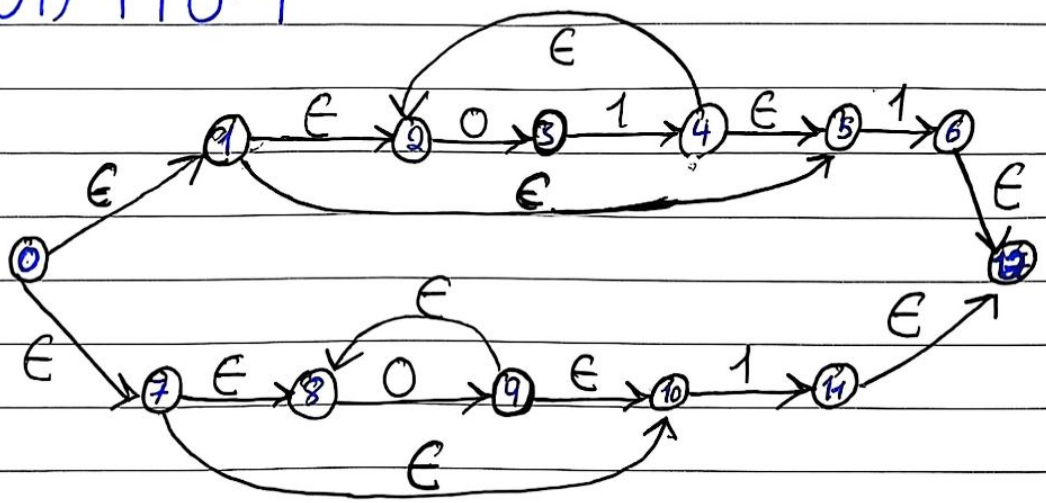
State	X	Y	Z
A	B	∅	C
B	A	D	C
C	A	∅	C
D	A	D	C

③ Splitting Table:

Umm... it's optimized

* Convert from NFA to DFA

$(01)^*1 \mid 0^*1$



ϵ-closure:

O = {0, 1, 2, 5, 7, 8, 10} ← State A

mov(A, 0) = {3, 9}

ϵ-closure = {3, 8, 9, 10} ← State B

mov(A, 1) = {6, 11}

ϵ-closure = {6, 11, 12} ← State C

mov(B, 0) = {9}

ϵ-closure = {8, 9, 10} ← State D

mov(B, 1) = {4, 11}

ϵ-closure = {2, 4, 5, 11, 12} ← State E

mov(C, 0) = ∅

mov(C, 1) = ∅

mov(D, 0) = {9} ← State D

mov(D, 1) = {11}

ϵ-closure = {11, 12} ← State F

mov(E, 0) = {3}, ϵ-closure = {3} ← State G

mov(E, 1) = {6}, ϵ-closure = {6, 12} ← State H

mov(F,0) = ∅

mov(F,1) = ∅

mov(G,0) = ∅

mov(G,1) = {4} ~~← State I~~, ε-closure = {2,4,5} ← State I

mov(H,0) = ∅

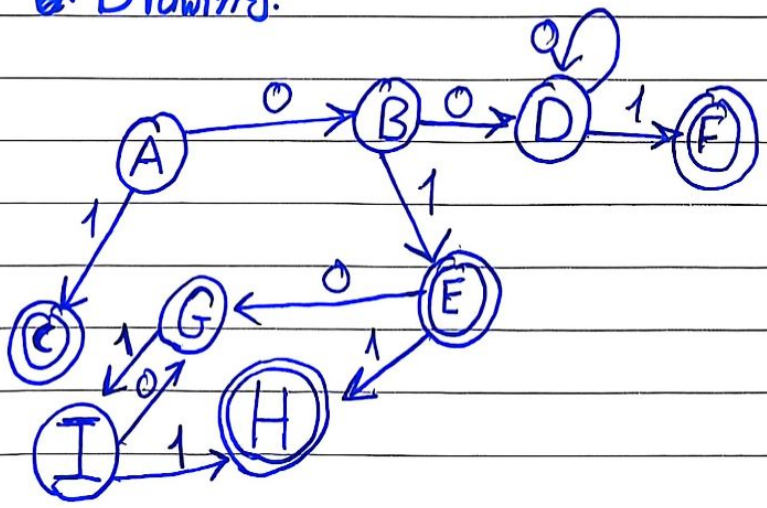
mov(H,1) = ∅

mov(I,0) = ~~∅~~ {3} ← G

mov(I,1) = {6} ~~H~~ ε-closure = {6,12,} ← State (H)

State	0	1
A	B	C
B	D	E
(C)	∅	∅
D	D	F
E	G	H
F	∅	∅
G	∅	I
(H)	∅	∅
I	G	H

DFA or Drawing:



DFA optimization:

① acceptance = {C, E, F, H}

non-acceptance = {A, B, D, G, I}

② Splitting Table:

	0	1
{C, E, F, H}, {A, B, D, G, I}	{C, F, H} {E} {A, B, D, G, I}	{A, D, I} {E} {C, F, H} {B} {G}
{A D I		

☆ خد بالك من الـ ε اللي مستخبية جوا خالص

☆ البتاع اللي أخد بالي أنه في نهاية Rule إياه عشان ممكن يكون بيعمل Recursive

☆ من الدوار بتاع الـ ε طالما الـ ε في final Path مثلاً لما أخد بالي

$$A \rightarrow aXb$$

First Set:

ده أول terminal بقابلها وأنا بملأ الـ Rule إياه

First (A): {a}

First (X): ∅ // X مفيش Rule ليها أو مفيش Non-ter. لونى الـ Rule الـ X

Follow Set:

ده أول terminal بقابلها جوا الـ RULE اللي أنا بعديها وأنا !

Follow (A): ∅ // A مش موجودة فى أى Rule تانية أو فى آخر Char فى الـ Rule

Follow (X): { b }

Example: Imp. Notes (In Follow Set):

- لو هو Start Symbol هحط فى الـ Follow set بدل $ عشان لما أجى أعمل String مبقاش عايز يخلص
- لو بعده Terminal هاخده على طول
- لو بعده Non-Terminal هاخد الـ First بتاعه (اللي هو الـ Non terminal) على اللام

☆ لو وقفت فى نهاية Rule بشوف الـ Follow بتاع الـ Follow اللي فيها وأخده Take it

ممكن تقفى نهاية Rule لو اللي بعديها تقدر تتحول لـ ε

☆ تعويض الـ ε بنشوف هيأثر إزاي (مستخبية)

☆ كل rule أو تعويضة لازم تبقى unique

$E \rightarrow TE'$

$E' \rightarrow +TE'$

$F \rightarrow (E) \mid id$

↳ TE'

★ Left Recursion بيحل Left factoring تمام شيا

Left Recursion: (how to Solve)

Exp → Exp + Term | Exp - Term | Term

↳ Exp → ~~+Term Exp' Exp Term~~ Term Exp'

Exp' → + Term Exp' | - Term Exp' | ∈

Indirect Left Recursion: (Determination)

A → Ba | Aa | c ★ لما أعوض بـ Ba هلاقي نفسي وعندنا Bb left recursion

B → Bb | Ab | d ★ لما أعوض بـ Ab هلاقي نفسي وعندنا Aa left recursion

Solution: (we solve the first left Recursion)

A → BaA' | cA'

A' → aA' | ∈

~~B → Abb' | dB~~ ⇒ B → Bb | BaA' | cA' | d

~~B' = bB' | ∈~~ B → ~~Abb' | dB~~ | dB' | cA'B' ~~| ∈~~

B' → bB' | aA'B' | ∈

1

$X \to XZ0 \mid X1 \mid X12 \mid 1$

$Y \to Y2 \mid YZ4 \mid 2$

$Z \to 0$

$R \to 3$

first: left ~~factoring~~ recursion problem

$X \to 1X'$

$X' \to Z0X' \mid 1X' \mid 12X' \mid \epsilon$

$Y \to 2Y'$

$Y' \to 2Y' \mid Z4Y' \mid \epsilon$

$Z \to 0$

$R \to 3$

first(X): {1}	follow(X): {$}
first(X'): {1, 0, ε}	follow(X'): {$}
first(X''): {1, 0, ε, 2}	follow(X''): {$}
first(Y): {2}	follow(Y): { }
first(Y'): {2, 0, ε}	follow(Y'): ∅
first(Z): {0}	follow(Z): {4, 0}
first(R): {3}	follow(R): ∅

Second: left factoring problem

$X \to 1X'$

$X' \to 1X'' \mid Z0X' \mid \epsilon$

$X'' \to X'12X'$

$Y \to 2Y'$

$Y' \to 2Y' \mid Z4Y' \mid \epsilon$

$Z \to 0$

$R \to 3$

2

$E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid id$

Augmented grammar

$E' \to E$

$E \to E+T$

$E \to T$

$T \to T*F$

$T \to F$

$F \to (E)$

$F \to id$

first(E) → {(, id}

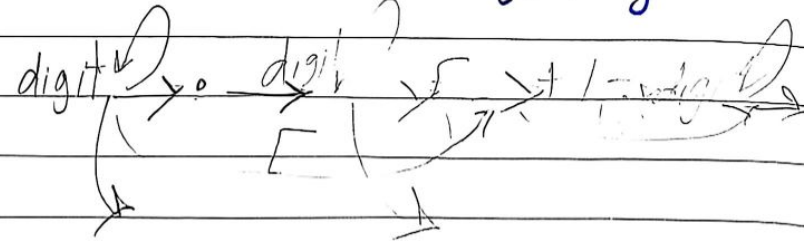
first(T) → {(, id}

first(F) → {(, id}

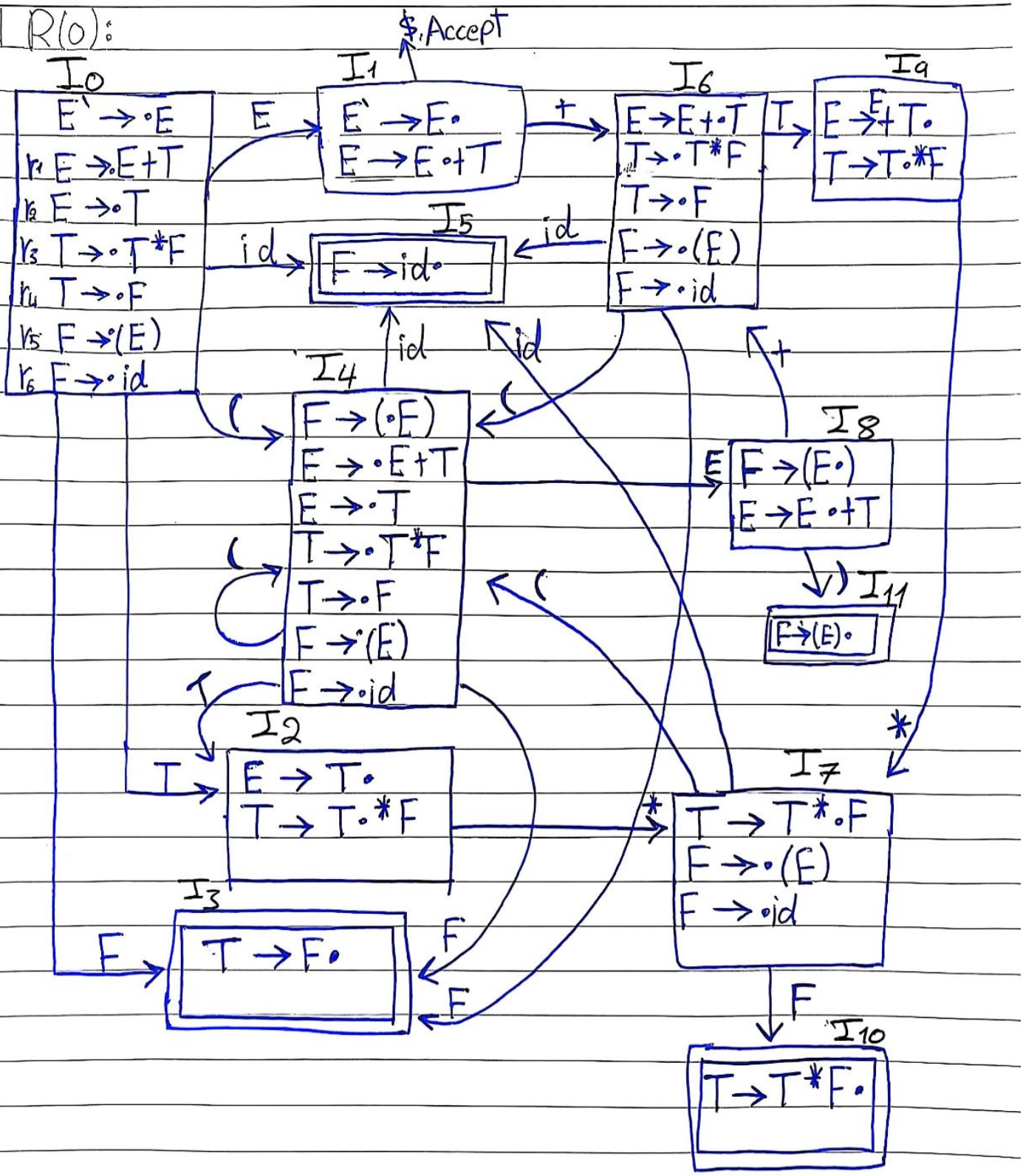
follow(E) → {), +, $}

follow(T) → {*,), +, $}

follow(F) → {*,), +, $}



LR(0):


$, Accept
I0
E' → ·E
r1 E → ·E+T
r2 E → ·T
r3 T → ·T*F
r4 T → ·F
r5 F → ·(E)
r6 F → ·id
I1
E' → E·
E → E·+T
I6
E → E+·T
T → ·T*F
T → ·F
F → ·(E)
F → ·id
I9
E → E+T·
T → T·*F
I5
F → id·
I4
F → (·E)
E → ·E+T
E → ·T
T → ·T*F
T → ·F
F → ·(E)
F → ·id
I8
F → (E·)
E → E·+T
I11
F → (E)·
I2
E → T·
T → T·*F
I3
T → F·
I7
T → T*·F
F → ·(E)
F → ·id
I10
T → T*F·
E
T
F
id
(
)
+
*

Action GO-To Table

	id	+	*	(	)	$	E	T	F
0	S5			S4			1	2	3
1		S6				accept	/	/	/
2		r2	S7		r2	r2	/	/	/
3		r4	r4		r4	r4	/	/	/
4	S5			S4			8	2	3
5		r6	r6		r6	r6	/	/	/
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		r1	S7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

لوفي قاعدة
في منتهية بـ follow الـ reduce بـ

Non Predictive Parsing Table

	id	+	*	(	)	$
E	TE"			TE"		
E"		+TE"			ϵ	ϵ
T	FT"			FT"		
T"		ϵ	*FT"		ϵ	ϵ
F	id			(E)		

لوفي كل ϵ بتيجي من الـ follow الـ
لازم أول الـ left recursion الأول left factoring ،

$E \rightarrow TE''$
$E'' \rightarrow +TE'' \mid \epsilon$
$T \rightarrow FT''$
$T'' \rightarrow *FT'' \mid \epsilon$
$F \rightarrow (E) \mid id$

first(E): {(, id}	follow(E): {), $}
first(E"): {+, ϵ}	follow(E"): {), $}
first(T): {(, id}	follow(T): {+,), $}
first(T"): {*, ϵ}	follow(T"): {+,), $}
first(F): {(, id}	follow(F): {*, +,), $}

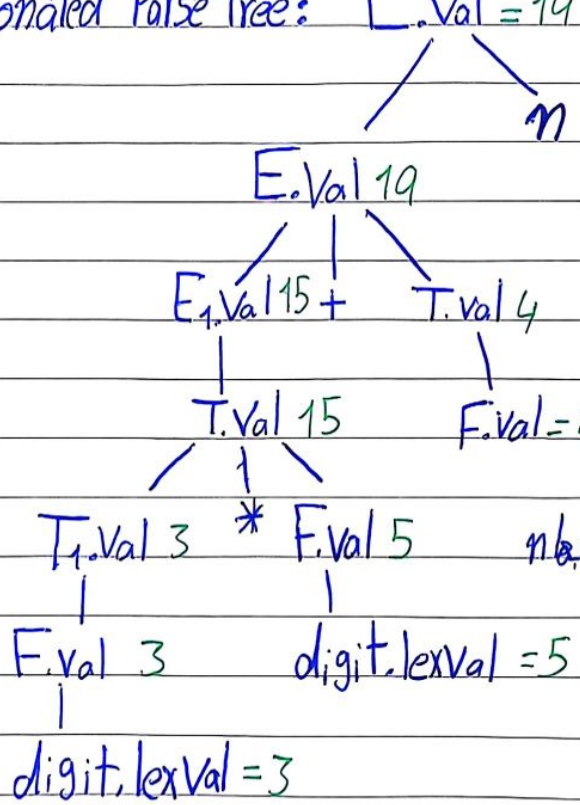
Moves of an LR Table

Lecture 11

⇒ Syntax directed translation: → It evaluates the expr

↳ This is not parsing → Its translation (Translates the Parse tree) and gives a semantic rule on the Production rule. (

Annonated Parse tree:



Note:
n is like $
الحاجة اللي هاخد الValue
lexval بتاعتها علطول قصيها
E is like E1
it means a value in the past

في الـ Semantic Rules أنا بهتم بالحاجة اللي هعملها Evaluation

في الـ Tree أقدر أكتب n أو أسيبها بس بهتم بقيمتها

وخلاص.

Types of Attributes:

↳ Synthesized: takes it from children

↳ Inherited: takes it from parent and Sibling nodes.

Production في الـ rules بتعبر عن قاعدة واحدة في الـ Semantic rules فلاقي ان في الـ